

A CLOSER LOOK AT POWER SAMPLING

Mauricio Barba da Costa¹ Ryan Cotterell²

¹MIT ²ETH Zürich

ABSTRACT

Recent advances in language model reasoning capabilities have been driven largely by the use of reinforcement learning with verifiable rewards (RLVR) in post-training pipelines. However, recent work has suggested that, at least with an open-source compute budget, RLVR may improve language-model performance primarily by increasing the probability of sequences that already have high probability under the base model. This theory has motivated the creation of training-free, test-time methods for sharpening the base model distribution based on the *power distribution*: a global tempering of the language model distribution that up-weights high-likelihood sequences. Prior work has shown that some algorithms that sample approximately from the power distribution can achieve strong accuracy for certain models and benchmarks. In this paper, we create a novel exact sampler for the power distribution and devise experiments for probing the power distribution. Our experiments show that exact samples from the power distribution recover many of the unstable behaviors observed in likelihood-maximizing decoding strategies. The power distribution improves accuracy for some models and benchmarks and drastically degrades accuracy on others, challenging the notion that performance gains observed with RLVR can be attributed to its use. Therefore, past gains observed from approximate sampling from the power distribution must be attributed to their sampling strategy rather than to the underlying target distribution.

1 INTRODUCTION

Early advances in language-model reasoning have been driven by large-scale next-token prediction pretraining (Kaplan et al., 2020; Brown et al., 2020). More recent advances are the result of post-training with reinforcement learning with verifiable rewards (RLVR) (Shao et al., 2024; DeepSeek, 2025; OpenAI, 2024). Yet, post-training is costly: it requires extensive rollouts and often relies on data that are difficult to collect, such as human judgments or synthetic data that are not publicly available. Scholars argue that RLVR may be worth this high cost because it enables models to acquire capabilities absent from the base model. However, a competing hypothesis holds that, at least at the compute budgets available to open-source projects (Shao et al., 2025), these gains may not involve new capabilities. Under this view, post-training raises benchmark accuracy primarily by concentrating probability mass on completions to which the base model already assigns relatively high probability (Yue et al., 2025; Shao et al., 2025; He et al., 2025; Huang et al., 2025). This is known as the *distribution sharpening* hypothesis.

The hypothesis becomes actionable once the sharpened distribution is given an explicit form. Karan & Du (2026) propose the *power distribution*, a global tempering of the base model that raises the probability of each completion to a power $\alpha > 1$ and renormalizes over complete sequences. Because this distribution is defined entirely in terms of the base model, it can be targeted at test-time by approximate inference, without rewards or further training. Any algorithm that samples approximately from the power distribution with test-time compute is known as *power sampling*. If exact power sampling captures the effect of RLVR on language model reasoning, part of the cost of post-training could be replaced by test-time computation, and approximate inference would constitute an additional axis for scaling capabilities (Karan & Du, 2026). As such, prior work has sought more computationally efficient algorithms for converging to the power distribution, ranging from Metropolis–Hastings (Karan & Du, 2026) to sequential Monte Carlo and related methods (Ji et al., 2026; Azizi et al., 2026; Zhou et al., 2026; Meng et al., 2026), and has reported improved benchmark accuracy.

Yet, these results leave open whether the improvements come from the power distribution itself or from the approximation method used to reach it. In this paper, we separate the power distribution from the approximation method using an exact sampler for the power distribution. Across four models and five mathematics and code benchmarks, we find that the power distribution at $\alpha = 4$ exhibits many behaviors of its $\alpha = \infty$ limit, which places all of its mass on the completions with the highest likelihood. Its accuracy closely tracks that of beam search, so it rises or falls with the quality of the highest-likelihood completions. The power distribution therefore reproduces the behavior of likelihood-maximization decoding rather than that of RLVR.

We make the following contributions:

An exact sampler for the power distribution. We present a rejection sampler that produces exact samples from the power distribution when certain conditions are met (Section 3.1). To increase the number of problems on which it accepts a sample, we use a highly vectorized GPU implementation. The rejection sampler yields samples on over 57% of the problems we evaluate. Because these samples come from the power distribution itself, they isolate the power distribution’s properties from artifacts of test-time approximate inference and of training dynamics involved in fine-tuning a policy toward it (Mittal et al., 2026).

Exact power sampling resembles likelihood-maximization decoding. Across four models and five benchmarks, accuracy under the power distribution at $\alpha = 4$ closely tracks that of best-first beam search (Table 1). Importance-sampling estimates show that this behavior emerges at small values of α (Figure 1). We observe that the log-likelihoods of completions output by the power distribution closely resemble those of completions found by beam search (Figure 2), and that completions under the power distribution shorten to lengths observed in completions obtained with beam search (Figure 4). On benchmarks, we observe that the power distribution inherits the accuracy of the high-likelihood completions, improving on autoregressive sampling in some settings and degrading it in others.

2 PRELIMINARIES

Completions and the language-model distribution. Let Σ be a token vocabulary containing an end-of-sequence token EOS. We write Σ^* for the set of finite completions $\mathbf{y} = (y_1, \dots, y_t)$ that end in EOS and contain no earlier EOS token. A prompt $\mathbf{x}$ is fixed conditioning context and is not part of the completion. The base language model defines next-token probabilities and hence the completion probability

$$p(\mathbf{y} \mid \mathbf{x}) = \prod_{i=1}^t p(y_i \mid \mathbf{x}, y_{<i}), \quad (1)$$

where $y_{<i} = (y_1, \dots, y_{i-1})$. The product includes the probability of EOS. We assume termination with probability one, so that $\sum_{\mathbf{y} \in \Sigma^*} p(\mathbf{y} \mid \mathbf{x}) = 1$. Autoregressive sampling draws each token from these conditional probabilities at temperature 1 until EOS is emitted.

The power distribution. For $\alpha \geq 1$, the power distribution over completions is

$$p_\alpha(\mathbf{y} \mid \mathbf{x}) = \frac{p(\mathbf{y} \mid \mathbf{x})^\alpha}{Z_\alpha(\mathbf{x})}, \quad Z_\alpha(\mathbf{x}) = \sum_{\mathbf{y}' \in \Sigma^*} p(\mathbf{y}' \mid \mathbf{x})^\alpha. \quad (2)$$

At $\alpha = 1$, this is the base-model distribution. For $\alpha > 1$, it increases the relative probability of higher-probability completions. This normalization is over complete outputs and generally differs from applying a temperature of $1/\alpha$ separately at each token.

The argmax completion. An argmax completion is any globally most probable completion,

$$\mathbf{y}^* \in \arg \max_{\mathbf{y} \in \Sigma^*} p(\mathbf{y} \mid \mathbf{x}). \quad (3)$$

Thus $p(\mathbf{y}^* \mid \mathbf{x})$ bounds the probability of every completion for the prompt. The same completions maximize p_α for every $\alpha \geq 1$. As $\alpha \rightarrow \infty$, the power distribution $p_\alpha(\mathbf{y} \mid \mathbf{x})$ converges to a distribution that assigns 0 to non-argmax completions and uniformly weights the argmax completions.

Beam search. Because Σ^* is infinite, exact computation of Equation 3 is generally intractable. To approximate it, we use a variant of beam search based on best-first beam search (Meister et al., 2020). The algorithm is simple: it iteratively pops the highest-likelihood sequence from the priority queue, creates a list of candidates by concatenating the sequence with all of the tokens that can be sampled next, then inserts these candidates into the priority queue ranked by likelihood. The algorithm proceeds in this manner until either a timeout is reached or an EOS-terminated sequence is popped from the queue. If an EOS-terminated sequence is popped from the queue, then it must be the highest-likelihood sequence in the language model’s probability distribution; all other sequences in the language model distribution are seeded by a sequence that is in the priority queue at the time that the EOS-terminated sequence is popped and concatenating a token to a sequence can only lower its likelihood. Though finding the highest-likelihood completion and certifying it is not guaranteed, the completions obtained by beam search typically have much higher likelihood than those produced by autoregressive decoding or temperature sampling even if they do not have an optimality certificate (Figure 2).

3 METHODS

We study the power distribution with three inference procedures, and compare them with autoregressive sampling from the base model at temperature 1 and beam search sampling. Rejection sampling (Section 3.1) produces exact samples from the power distribution on problems where an argmax completion can be certified and at least one proposal is accepted; it is the reference against which we judge the other procedures. As rejection-sampling estimates are only available on a subset of benchmark tasks, estimates reported below are averaged over problems where at least one rejection sample was available. Self-normalized importance sampling (Section 3.2) estimates expectations under the power distribution on every problem, including problems where rejection sampling produces no sample. To understand the sensitivity of the Metropolis–Hastings-based inference algorithm offered by Karan & Du (2026) to its initial starting point, we run two instantiations of their algorithm. First, we run their original Metropolis–Hastings algorithm. Then, we run the same algorithm but initialize it at a completion obtained with beam search.

3.1 EXACT SAMPLING WITH REJECTION SAMPLING

In principle, it is simple to derive a naive rejection sampler for drawing exact samples from the power distribution: simply draw samples from $\mathbf{y} \sim p(\cdot | \mathbf{x})$ via ordinary autoregressive sampling and accept with probability $\frac{p(\mathbf{y}|\mathbf{x})^\alpha}{p(\mathbf{y}|\mathbf{x})} = p(\mathbf{y} | \mathbf{x})^{\alpha-1}$. However, the low acceptance rate of this algorithm makes it untenable for practical use. Using the example displayed in Figure 5 as an illustration, the highest-likelihood completion for the problem has a probability of 1.4×10^{-4} . As this completion also maximizes $p(\cdot | \mathbf{x})^{\alpha-1}$, this is also the completion with the highest acceptance rate. Despite having the highest acceptance rate, this completion is accepted with probability 2.6×10^{-12} .

Our rejection sampling procedure is split into two stages that are intended to increase the number of accepted samples (Algorithm 1). First, our procedure searches for an argmax completion $\mathbf{y}^*$ (Equation 3) together with a certificate that no completion has a higher likelihood. Second, it draws proposals from the base model with temperature 1 and accepts each proposal $\mathbf{y}$ with probability $(p(\mathbf{y} | \mathbf{x})/p(\mathbf{y}^* | \mathbf{x}))^{\alpha-1}$. Accepted proposals are exact samples from p_α . Appendix A.1 gives a proof of the correctness of the algorithm. When a completion with a certificate of its optimality is obtained, the highest-likelihood completion is always accepted if it is proposed in the second stage. Thus, if 1,024 proposals are drawn from the base model in stage 2, the probability of proposing the argmax completion in the example above is $1 - (1 - 1.4 \times 10^{-4})^{1,024} = 0.13$. This means that for this particular benchmark problem and model, a lower bound for our algorithm’s acceptance rate is 13%.

To rapidly search for a certified argmax completion and propose samples, we implement both stages as batched GPU computations. Our search algorithm is a modified version of the best-first search algorithm (Meister et al., 2020) described in Section 2 that uses batch inference to search different parts of the distribution in parallel. Rather than popping just a single sequence from the priority queue as described above, we pop the top-128 sequences in parallel and compute the probabilities of their next tokens. To limit memory overhead on the GPU, we extend popped sequences with only

Algorithm 1 Exact rejection sampling from the power distribution.

```

1  def rejection_sample(model, x, alpha, k=256, b=128, timeout=180, B=1024):
2      """Exact samples from p_alpha, or [] if no argmax certificate."""
3      # Stage 1: certify an argmax completion with best-first search.
4      y_star = greedy_decode(model, x)
5      queue = PriorityQueue() # prefixes, most probable first
6      queue.push(EMPTY_PREFIX, logp=0.0)
7      certified, start = True, time.time()
8      while queue and time.time() - start < timeout:
9          for y in queue.pop_best(b): # b most probable prefixes, one batch
10             if model.logp(x, y) <= model.logp(x, y_star):
11                 continue
12             if y.rank == k: # rank of y's last token among parent's top-k
13                 certified = False # omitted tokens could lead higher
14             if y.is_complete():
15                 y_star = y
16             else:
17                 for a in model.top_k_next_tokens(x, y, k):
18                     queue.push(y + a, logp=model.logp(x, y + a))
19             if model.logp(x, y_star) >= queue.max_logp():
20                 break # nothing left in the distribution can beat y_star
21         if time.time() - start >= timeout:
22             certified = False
23         if not certified:
24             return []
25
26         # Stage 2: accept or reject proposals from the base model.
27         ys = model.sample(x, n=B, temperature=1.0) # in parallel
28         log_ratio = (alpha - 1) * (model.logp(x, ys) - model.logp(x, y_star))
29         log_u = np.log(np.random.rand(B))
30         return ys[log_u <= log_ratio]

```

their top-256 most-likely tokens. At the proposal stage, rather than sampling a single sequence at a time and checking if it is accepted, we generate batches of 1,024 proposals and return all completions that are accepted. Appendix A.2 gives more details on the percentage of problems where we are able to obtain a completion with a certificate of its optimality and the percentage of problems where the rejection sampler is able to generate a sample.

3.2 APPROXIMATION WITH SELF-NORMALIZED IMPORTANCE SAMPLING

Rather than producing samples, self-normalized importance sampling estimates expectations under the power distribution. For a prompt $\mathbf{x}$ and a statistic $f(\mathbf{x}, \mathbf{y})$, we draw N completions $\mathbf{y}_1, \dots, \mathbf{y}_N$ independently from the base model at temperature 1 and estimate $\mathbb{E}_{\mathbf{Y} \sim p_\alpha(\cdot|\mathbf{x})}[f(\mathbf{x}, \mathbf{Y})]$ by

$$\hat{\mathbb{E}}_\alpha^N[f | \mathbf{x}] = \sum_{i=1}^N \bar{w}_i f(\mathbf{x}, \mathbf{y}_i), \quad \bar{w}_i = \frac{p(\mathbf{y}_i | \mathbf{x})^{\alpha-1}}{\sum_{\ell=1}^N p(\mathbf{y}_\ell | \mathbf{x})^{\alpha-1}}. \quad (4)$$

The weight $p(\mathbf{y} | \mathbf{x})^{\alpha-1}$ is the ratio of the unnormalized target $p(\mathbf{y} | \mathbf{x})^\alpha$ to the proposal $p(\mathbf{y} | \mathbf{x})$. This is a biased but consistent estimator of $\mathbb{E}_{\mathbf{Y} \sim p_\alpha(\cdot|\mathbf{x})}[f(\mathbf{x}, \mathbf{Y})]$. To estimate accuracy on a benchmark of prompts $\{\mathbf{x}_m\}_{m=1}^M$, we set $f(\mathbf{x}, \mathbf{y}) = 1$ if $\mathbf{y}$ is a correct solution to $\mathbf{x}$ and 0 otherwise, and average over prompts:

$$\hat{\mu}_\alpha = \frac{1}{M} \sum_{m=1}^M \hat{\mathbb{E}}_\alpha^N[f | \mathbf{x}_m]. \quad (5)$$

Other statistics, such as the histogram of completion lengths in Figure 4 and the fraction of completions of at most five tokens in Figure 3, are estimated in a similar way. We produce $N = 1,024$ proposals per prompt using batch inference.

3.3 METROPOLIS–HASTINGS WITH BEAM-SEARCH INITIALIZATION

We use the Metropolis–Hastings sampler from Karan & Du (2026) in our experiments. It builds a completion incrementally in blocks of S tokens. At stage j , it targets the power distribution restricted to completions of at most $L_j = jS$ tokens, extends the current completion to the new length cap, and performs a fixed number of Metropolis–Hastings updates. Each update chooses a position in the current completion uniformly at random, keeps the prefix before it, resamples the suffix from the base model at temperature $1/\alpha$, and accepts the result with the Metropolis–Hastings acceptance probability. We use blocks of $S = 192$ tokens and 10 updates per block.

We run this sampler from two initializations. The first sampler starts from an empty completion. It draws every proposed suffix at temperature $1/\alpha$, which concentrates on tokens that are probable at each step and rarely produces the improbable first tokens with which degenerate completions such as “ δt ” begin (Figure 5). Because such completions can carry much of the mass of the power distribution (Section 4.2), a chain that rarely proposes them may not reach the power distribution within a practical number of updates. The second initialization uses the same proposal distribution and incremental staging scheme, but is initialized from the output of best-first beam search. Nearly every proposal from this initial state produces a different, typically longer completion, so if the power distribution placed little mass near the beam-search output, the chain would have many opportunities to leave it. A chain that stays near its starting point despite these proposals therefore suggests that the power distribution concentrates there.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

We evaluate four language models—Qwen2.5-7B, Qwen3-8B-Base, Olmo-2-7B-SFT, and Olmo-3-7B-SFT—on mathematics (MATH500, AIME24, AIME25) and code generation (MBPP, HumanEval). The Qwen models connect our evaluation to prior work on sampling from the power distribution (Karan & Du, 2026). We select Olmo for its openly documented training data and procedures. We use its SFT checkpoints for their standardized chat templates.

We compare six methods. Two do not target the power distribution: autoregressive sampling from the full vocabulary at temperature 1, and beam search. The other four target the power distribution with $\alpha = 4$: rejection sampling, importance sampling, and Metropolis–Hastings from the empty completion and from the beam-search output. Both Metropolis–Hastings samplers perform 10 updates per 192-token block. All methods are evaluated on the full benchmarks except rejection sampling, which we evaluate only on problems with at least one accepted proposal; we report the number of such problems with each result. We give further details in Appendix ??.

4.2 ANALYSIS

Accuracy varies across models and benchmarks but is broadly consistent across inference methods. Table 1 shows benchmark accuracy across all models and benchmarks we experimented with. For each model and benchmark, the table shows the accuracy under autoregressive sampling, beam search, and the rejection-sampling, importance-sampling, and Metropolis–Hastings procedures described above. The table shows that the effects of the power distribution depend strongly on the model and benchmark, but rejection sampling, importance sampling, and beam-search-initialized Metropolis–Hastings often yield similar accuracy within a given setting. For example, all three methods substantially reduce Qwen2.5-7B’s accuracy on math benchmarks while improving it on MBPP. Figure 6 shows that these results also hold when the set of benchmark problems is restricted only to those where rejection sampling produces a sample. This suggests that restricting the results in Table 1 to only include the problems where a sample was produced with rejection sampling does not introduce a bias in our results. This agreement extends beyond accuracy: Figure 4 shows similar completion-length distributions for the three methods across most models and benchmarks, and Figures 2 and 3 show that, for the Qwen2.5 model, the log-likelihood histogram and the fraction of completions with 5 or fewer tokens, respectively, are similar across the three inference methods while differing substantially from autoregressive decoding. Agreement across distinct inference

Table 1: **Model accuracy (%) under different sampling algorithms.** The methods are autoregressive sampling (AR), Metropolis–Hastings from the empty completion (MH), beam search (BS), rejection sampling (RS), self-normalized importance sampling (SNIS), and Metropolis–Hastings from the beam-search output (MH-BS). RS includes only problems with a saved argmax certificate and at least one accepted proposal, and its brackets give qualifying/full benchmark problem counts.

Model	Method	MATH500	AIME24	AIME25	MBPP	HumanEval
Qwen2.5-7B	AR	44.79	3.68	1.00	38.02	23.21
	MH	66.60	16.67	6.67	71.21	52.44
	BS	5.20	0.00	0.00	63.81	20.73
	RS	3.94 [165/500]	0.00 [10/30]	0.00 [11/30]	69.16 [231/257]	20.83 [163/164]
	SNIS	4.64	0.07	0.00	65.42	20.37
	MH-BS	5.40	0.00	0.00	64.20	20.12
Qwen3-8B-Base	AR	58.92	7.64	5.96	9.52	12.40
	MH	78.00	16.67	16.67	24.12	59.76
	BS	68.80	10.00	10.00	3.11	32.93
	RS	77.46 [71/500]	0.00 [1/30]	0.00 [1/30]	5.50 [252/257]	35.54 [161/164]
	SNIS	50.47	0.00	3.33	5.17	35.06
	MH-BS	67.00	13.33	10.00	5.06	34.76
Olmo-3-7B-SFT	AR	62.11	5.09	4.63	53.63	61.86
	MH	73.60	10.00	13.33	68.48	76.22
	BS	37.60	3.33	6.67	68.87	71.95
	RS	45.48 [158/500]	0.00 [6/30]	0.00 [4/30]	69.77 [250/257]	73.03 [163/164]
	SNIS	40.38	3.33	0.00	68.58	72.67
	MH-BS	35.40	0.00	6.67	68.48	73.17
Olmo-2-7B-SFT	AR	12.29	0.38	0.17	29.40	20.62
	MH	17.40	0.00	0.00	48.25	38.41
	BS	9.40	0.00	0.00	42.02	31.71
	RS	14.16 [180/500]	0.00 [4/30]	0.00 [5/30]	41.84 [257/257]	31.90 [162/164]
	SNIS	10.08	0.00	0.00	41.88	31.57
	MH-BS	8.80	0.00	0.00	40.86	29.88

procedures and multiple distributional statistics suggests that the estimates given by importance sampling and the Metropolis–Hastings sampler are close to their actual values.

Accuracy under the power distribution largely matches that of beam search. Figure 1 shows that the importance-sampling accuracy estimates change most sharply near $\alpha = 1$ and largely plateau for $\alpha > 2$. Table 1 shows that accuracy under the power distribution with $\alpha = 4$ is often close to that of beam search. These results suggest that modest sharpening is sufficient for the power distribution to recover behaviors observed in likelihood-maximizing decoding strategies. As beam search is one decoding strategy for sampling near the mode of the base model distribution, samples drawn exactly from the power distribution exhibit properties of completions generated with beam search. This is further evidenced by Figure 2, which shows that the log-likelihoods of the three methods tightly concentrate near zero, while the log-likelihoods of completions drawn via autoregressive decoding remain spread out.

The power distribution leads to unstable completions. Figure 3 shows that, for Qwen2.5-7B, completions of at most five tokens account for 87–92% of the mass on MATH500 under rejection

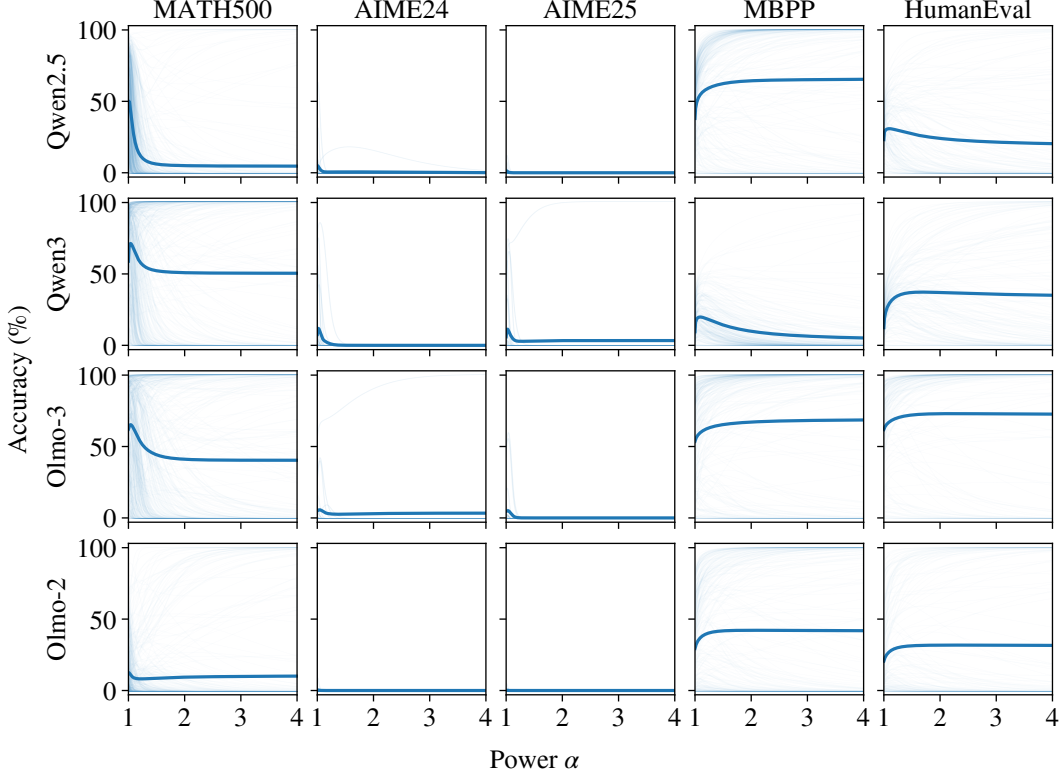


Figure 1: **Estimated accuracy (%) under the power distribution as a function of α .** These accuracy estimates use self-normalized importance sampling with 1,024 temperature-1 completions per problem (Equation 4). Rows correspond to models (Qwen2.5-7B, Qwen3-8B-Base, Olmo-3-7B-SFT, Olmo-2-7B-SFT) and columns to benchmarks. Faint curves show individual problems; bold curves show benchmark averages, with weights normalized within each problem.

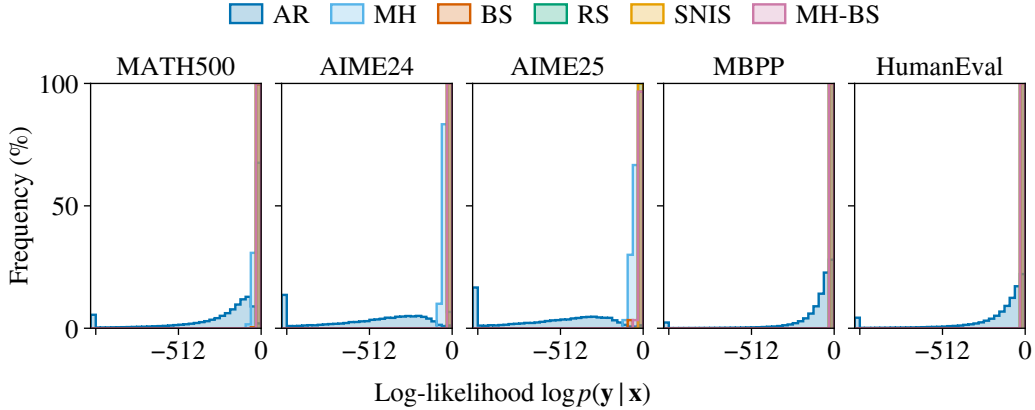


Figure 2: **Log-likelihood of returned completions for Qwen2.5-7B.** Distribution of $\log p(\mathbf{y} | \mathbf{x})$; the leftmost bin collects completions below $-1,024$ (method abbreviations as in Table 1).

sampling, importance sampling, and Metropolis–Hastings from the beam-search output, compared with less than 1% under autoregressive sampling, and that almost none of them are correct. This concentration explains most of the decline in accuracy from roughly 45% to 5% in Table 1.

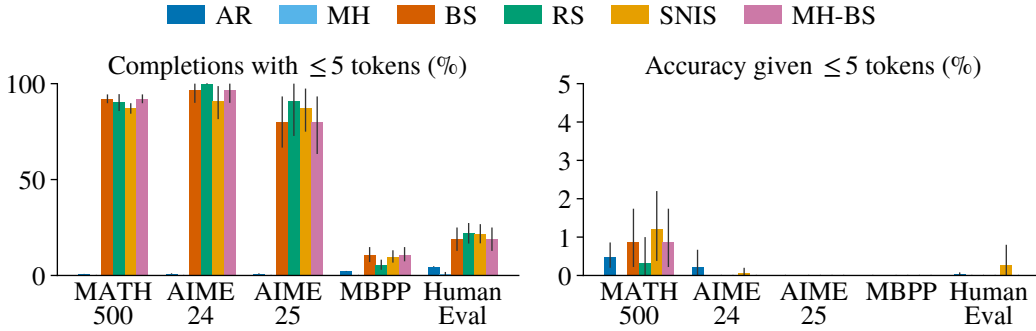


Figure 3: **Short completions for Qwen2.5-7B.** Left: percentage of completions with at most five tokens. Right: accuracy among those completions (MH produces none on four benchmarks). Error bars are 95% bootstrap intervals over problems.

Prior work has shown that the highest-likelihood completions in a language model are unstable, often exhibiting degeneracy and a short length bias (Holtzman et al., 2020; Stahlberg & Byrne, 2019; Yoshida et al., 2024). At $\alpha = \infty$ the power distribution places all of its mass on the highest-likelihood completions. Figure 5 (Appendix C) illustrates why these completions are often degenerate. For the prompt “How many positive whole-number divisors does 196 have?”, most of the probability mass of Qwen2.5-7B lies on correct solutions, but that mass is divided among many distinct phrasings of the solution, none of which is individually probable (Holtzman et al., 2021). The most probable completion, “ô”, begins with an improbable token but ends immediately with EOS, and so avoids accumulating a lower likelihood with each sampled non-EOS token.

The power distribution shortens sequence lengths. In addition to increasing the frequency of degenerate completions with 5 or fewer tokens, Figure 4 (Appendix B) shows that Qwen2.5-7B completions get shorter on average across all five benchmarks we experimented with. Autoregressive sampling produces a broad range of completion lengths, whereas rejection sampling, importance sampling, and Metropolis–Hastings from the beam-search output concentrate most of their mass in the shortest length bin. This shift occurs across all five benchmarks, indicating that global sharpening consistently favors shorter completions in these experiments. This observation contrasts with RL, which tends to lengthen completions (Shao et al., 2024).

Initializing Metropolis–Hastings at the beam-search output makes it resemble rejection and importance sampling. We observe in Table 1 and Figures 4, 3, and 6 that the distributional statistics of the Metropolis–Hastings sampler deviate significantly from those of rejection sampling and importance sampling when it is initialized from the empty string, but they match closely when the sampler is initialized at the completion produced by beam search. Especially notable is that, while rejection sampling and importance sampling often hurt benchmark accuracy, Metropolis–Hastings with empty initialization nearly always improves accuracy across models and benchmarks. Because the power distribution with $\alpha = 4$ resembles beam search, initializing the Metropolis–Hastings sampler at a completion obtained with beam search may be treated as initializing Metropolis–Hastings on a completion produced by a Monte Carlo chain that has already converged. We conclude from this that the initialization of Metropolis–Hastings plays a key role in its improved accuracy across benchmarks and models.

5 RELATED WORK

Power sampling. Karan & Du (2026) introduce the power distribution as a training-free target for reasoning and approximate it with Metropolis–Hastings. Subsequent work proposes more efficient approximations based on sequential Monte Carlo and related methods (Azizi et al., 2026; Ji et al., 2026; Meng et al., 2026; Zhou et al., 2026). These methods are similar in their use of an inference scheme to draw approximate samples from an exponentiated version of the base distribution.

Mode degeneracy in language models. Prior work has investigated the counterintuitive observation that decoding strategies that maximize likelihood often lead to degenerate samples like the ones observed above, and several hypotheses have been proposed for why this behavior occurs. Holtzman et al. (2020) show that the tail of a language model’s token distribution is unreliable and leads to degenerate patterns such as short sequences and repetitive completions. Holtzman et al. (2021) argue that completions in a language model that represent the *correct* answer do not have the highest likelihoods because they fragment the total probability of outputting such a correct answer. Finlayson et al. (2024); Eikema & Aziz (2020) argue that the root cause of the instability of the tails of language models’ token distributions is numerical error that accrues for small outputs of the softmax.

Distribution sharpening and RLVR. Several studies argue that RLVR primarily reweights outputs that the base model can already produce. Yue et al. (2025) find that RLVR improves pass@1 while base models attain higher pass@ k at large k , Shao et al. (2025) report that RLVR with random or incorrect rewards can still improve accuracy for some model families, and He et al. (2025) show that GRPO preferentially reinforces already-probable solutions. Yet other prior work has found evidence against the distribution sharpening hypothesis. In a controlled chess setting, Shen et al. (2026) estimate an effective power coefficient α by regressing the centered log-probabilities of a GRPO-trained move policy on those of the supervised policy from which it was initialized, since exact power sharpening would make this relationship linear with slope α . The fitted slope increases during RL, but the fit explains only part of the variance and the per-state slopes vary substantially, indicating that RL is not well described by a single power transformation of the initial policy.

Most similar to our work is the approach adopted by Mittal et al. (2026), who compare sharpening-based and task-reward-based objectives within a single RL fine-tuning framework. They fine-tune policies toward two sharpened objectives: one maximized by the argmax completion and one maximized by the power distribution. The power distribution enters their study only through policies fine-tuned toward the second objective. Because this policy is trained by stochastic, non-convex optimization and need not reach the optimum of its objective, their findings reflect the training algorithm as well as the target. These policies often reach accuracies that differ substantially from those of the first objective, even at values of α at which we find the power distribution to closely resemble the argmax completion (Section 4.2); for Qwen2.5-3B on MATH500, for example, the final checkpoint reaches 61.3% at $\alpha = 2$ against 19.6% for the first objective. Unlike Mittal et al. (2026), we study the power distribution without training: rejection sampling yields exact samples wherever an argmax completion can be certified, which isolates properties of the power distribution from artifacts of both test-time approximate inference and RL fine-tuning.

6 CONCLUSION

In this paper, we investigate the hypothesis that the power distribution can explain reasoning gains associated with RLVR post-training. We probe the power distribution with a variety of techniques and find that even with moderate global tempering (low $\alpha > 1$), results closely resemble likelihood-maximization decoding (equivalent to the power distribution with $\alpha = \infty$), improving for some models and benchmarks and markedly degrading on others. The distributions of sequence lengths for the power distribution and beam search decoding are nearly identical, and the frequency of degenerate completions increases under both. Based on these findings, we conclude that the power distribution does not recover RLVR-like behaviors. Thus, prior methods that showed improved accuracy across benchmarks and models with approximate power sampling benefited from artifacts of their approximate inference algorithm. What latent properties these sampling algorithms exploit remains a worthwhile open question, as these properties may be leveraged for developing cheaper verifier-free decoding strategies that improve base model benchmark accuracy.

AI USE STATEMENT

In this work, we used generative AI tools to assist with implementing portions of our experiments and to review the presentation of our methodology and experiments. We additionally used generative AI tools to edit the manuscript for clarity and concision and to revise L^AT_EX formatting and produce figures. We did not use generative AI to generate experimental data or conduct experiments. All

AI-assisted code and manuscript revisions were reviewed by the authors; the code was tested, experiments were run by the authors, and reported values were checked against the underlying results. We take responsibility for the final content of this work, including all AI-assisted text, claims, code, and artifacts.

REFERENCES

- Seyedarmin Azizi, Erfan Baghaei Potraghloo, Mino Ahmadi, Souvik Kundu, and Massoud Pedram. Power-smc: Low-latency sequence-level power sampling for training-free llm reasoning, 2026. URL <https://arxiv.org/abs/2602.10273>.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- DeepSeek. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645 (8081):633–638, September 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL <http://dx.doi.org/10.1038/s41586-025-09422-z>.
- Bryan Eikema and Wilker Aziz. Is MAP decoding all you need? the inadequacy of the mode in neural machine translation. In Donia Scott, Núria Bel, and Chengqing Zong (eds.), *Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020*, pp. 4506–4520. International Committee on Computational Linguistics, 2020. doi: 10.18653/V1/2020.COLING-MAIN.398. URL <https://doi.org/10.18653/v1/2020.coling-main.398>.
- Matthew Finlayson, John Hewitt, Alexander Koller, Swabha Swayamdipta, and Ashish Sabharwal. Closing the curious case of neural text degeneration. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=dONpC9GL1o>.
- Andre Wang He, Daniel Fried, and Sean Welleck. Rewarding the unlikely: Lifting GRPO beyond distribution sharpening. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pp. 25548–25560. Association for Computational Linguistics, 2025. doi: 10.18653/V1/2025.EMNLP-MAIN.1298. URL <https://doi.org/10.18653/v1/2025.emnlp-main.1298>.
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*, 2020. URL <https://openreview.net/forum?id=rygGQyrFvH>.
- Ari Holtzman, Peter West, Vered Shwartz, Yejin Choi, and Luke Zettlemoyer. Surface form competition: Why the highest probability answer isn’t always right. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, pp. 7038–7051. Association for Computational Linguistics, 2021. doi: 10.18653/V1/2021.EMNLP-MAIN.564. URL <https://doi.org/10.18653/v1/2021.emnlp-main.564>.
- Audrey Huang, Adam Block, Dylan J. Foster, Dhruv Rohatgi, Cyril Zhang, Max Simchowitz, Jordan T. Ash, and Akshay Krishnamurthy. Self-improvement in language models: The sharpening mechanism. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL <https://openreview.net/forum?id=WJaUkwci9o>.
- Xiaotong Ji, Rasul Tutunov, Matthieu Zimmer, and Haitham Bou-Ammar. Scalable power sampling: Unlocking efficient, training-free reasoning for llms via distribution sharpening. *CoRR*, abs/2601.21590, 2026. doi: 10.48550/ARXIV.2601.21590. URL <https://doi.org/10.48550/arXiv.2601.21590>.

- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *CoRR*, abs/2001.08361, 2020. URL <https://arxiv.org/abs/2001.08361>.
- Aayush Karan and Yilun Du. Reasoning with sampling: Your base model is smarter than you think. In Carl Vondrick, Bharath Hariharan, Colin Raffel, Lerrel Pinto, Diyi Yang, and Aleksandra Faust (eds.), *The Fourteenth International Conference on Learning Representations, ICLR 2026, Rio de Janeiro, Brazil, April 23-27, 2026*. proceedings.iclr.cc, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/01498f17142547dadb60999c39bd0a09-Abstract-Conference.html.
- Clara Meister, Ryan Cotterell, and Tim Vieira. Best-first beam search. *Trans. Assoc. Comput. Linguistics*, 8:795–809, 2020. doi: 10.1162/TACL_A_00346. URL https://doi.org/10.1162/tacl_a_00346.
- Zibin Meng, Peng Xie, and Kani Chen. Depth-entropy guided sampling for training-free llm reasoning, 2026. URL <https://arxiv.org/abs/2607.09693>.
- Sarthak Mittal, Léo Gagnon, and Guillaume Lajoie. Beyond distribution sharpening: The importance of task rewards. *CoRR*, abs/2604.16259, 2026. doi: 10.48550/ARXIV.2604.16259. URL <https://doi.org/10.48550/arXiv.2604.16259>.
- OpenAI. OpenAI o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, Yulia Tsvetkov, Hannaneh Hajishirzi, Pang Wei Koh, and Luke Zettlemoyer. Spurious rewards: Rethinking training signals in RLVR. *CoRR*, abs/2506.10947, 2025. doi: 10.48550/ARXIV.2506.10947. URL <https://doi.org/10.48550/arXiv.2506.10947>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *CoRR*, abs/2402.03300, 2024. doi: 10.48550/ARXIV.2402.03300. URL <https://doi.org/10.48550/arXiv.2402.03300>.
- Jingyan Shen, Ang Li, Salman Rahman, Yifan Sun, Micah Goldblum, Matus Telgarsky, and Pavel Izmailov. Understanding reasoning from pretraining to post-training. *CoRR*, abs/2607.16097, 2026. doi: 10.48550/ARXIV.2607.16097. URL <https://doi.org/10.48550/arXiv.2607.16097>.
- Felix Stahlberg and Bill Byrne. On NMT search errors and model errors: Cat got your tongue? In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (eds.), *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3356–3362, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1331. URL <https://aclanthology.org/D19-1331/>.
- John von Neumann. Various techniques used in connection with random digits. In *Monte Carlo Method*, volume 12 of *National Bureau of Standards Applied Mathematics Series*, pp. 36–38. U.S. Government Printing Office, Washington, DC, 1951. URL https://mcnp-green.lanl.gov/pdf_files/InBook_Computing_1961_Neumann_JohnVonNeumannCollectedWorks_VariousTechniquesUsedinConnectionwithRandomDigits.pdf. Summary written by George E. Forsythe.
- Davis Yoshida, Kartik Goyal, and Kevin Gimpel. Map’s not dead yet: Uncovering true language model modes by conditioning away degeneracy. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pp. 16164–16215. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.ACL-LONG.855. URL <https://doi.org/10.18653/v1/2024.acl-long.855>.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? In Danielle Belgrave, Cheng Zhang, Laura N. Montoya, Hsuan-Tien Lin, Razvan Pascanu, Piotr Koniusz, Marzyeh Ghassemi, Nancy Chen, Iván Vladimir Meza Ruíz, and Arturo Loaiza-Bonilla (eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico , November 30 - December 5, 2025*, 2025. URL http://papers.nips.cc/paper_files/paper/2025/hash/537d5aa768c2d534016a4d06f87bc8fb-Abstract-Conference.html.

Felix Zhou, Anay Mehrotra, and Quanquan C. Liu. Reasoning with sampling: Cutting at decision points. *CoRR*, abs/2605.30327, 2026. doi: 10.48550/ARXIV.2605.30327. URL <https://doi.org/10.48550/arXiv.2605.30327>.

Max Zuo. mh-llm: A Metropolis–Hastings sampler for LLMs. <https://github.com/maxzuo/mh-llm>, 2025. Software, version 0.0.1.

A REJECTION SAMPLER

A.1 CORRECTNESS

Correctness follows from classical rejection sampling (von Neumann, 1951). Fix a prompt $\mathbf{x}$ and suppress conditioning on it. Let p be the distribution over completions ending at EOS or length T . A valid argmax certificate for $\mathbf{y}^*$ gives $p(\mathbf{y}) \leq p(\mathbf{y}^*)$ for every completion. For $\alpha > 1$, this implies

$$p(\mathbf{y})^\alpha \leq p(\mathbf{y}^*)^{\alpha-1} p(\mathbf{y}) = Mp(\mathbf{y}), \quad M = p(\mathbf{y}^*)^{\alpha-1}. \quad (6)$$

Thus Mp is an upper bound on the unnormalized target p^α , and rejection sampling with proposal p accepts each proposal $\mathbf{y}$ with probability

$$\frac{p(\mathbf{y})^\alpha}{Mp(\mathbf{y})} = \left(\frac{p(\mathbf{y})}{p(\mathbf{y}^*)} \right)^{\alpha-1}, \quad (7)$$

which is the rule in Algorithm 1. Independent proposals and uniform draws therefore give independent samples from p_α , and no normalizing constant is needed.

A.2 COVERAGE

Table 2: Problems (%) on which the search certifies an argmax completion, and on which the rejection sampler at $\alpha = 4$ additionally accepts at least one of the 1,024 proposals.

Model	Problems	MATH500	AIME24	AIME25	MBPP	HumanEval
Qwen2.5-7B	Certified argmax	98.4	100.0	93.3	100.0	100.0
	≥ 1 accepted	33.0	33.3	36.7	89.9	99.4
Qwen3-8B-Base	Certified argmax	39.0	20.0	23.3	100.0	100.0
	≥ 1 accepted	14.2	3.3	3.3	98.1	98.2
Olmo-3-7B-SFT	Certified argmax	47.4	40.0	26.7	98.8	100.0
	≥ 1 accepted	31.6	20.0	13.3	97.3	99.4
Olmo-2-7B-SFT	Certified argmax	86.0	76.7	66.7	100.0	100.0
	≥ 1 accepted	36.0	13.3	16.7	100.0	98.8

Table 3: Average number of accepted proposals per problem for the rejection sampler at $\alpha = 4$, out of 1,024 proposals, over all problems with a certified argmax.

Model	MATH500	AIME24	AIME25	MBPP	HumanEval
Qwen2.5-7B	0.44	0.43	0.46	13.06	7.51
Qwen3-8B-Base	0.73	0.17	0.14	45.32	28.79
Olmo-3-7B-SFT	1.99	0.75	0.88	173.67	168.95
Olmo-2-7B-SFT	0.99	0.17	0.25	79.37	62.18

B COMPLETION LENGTH

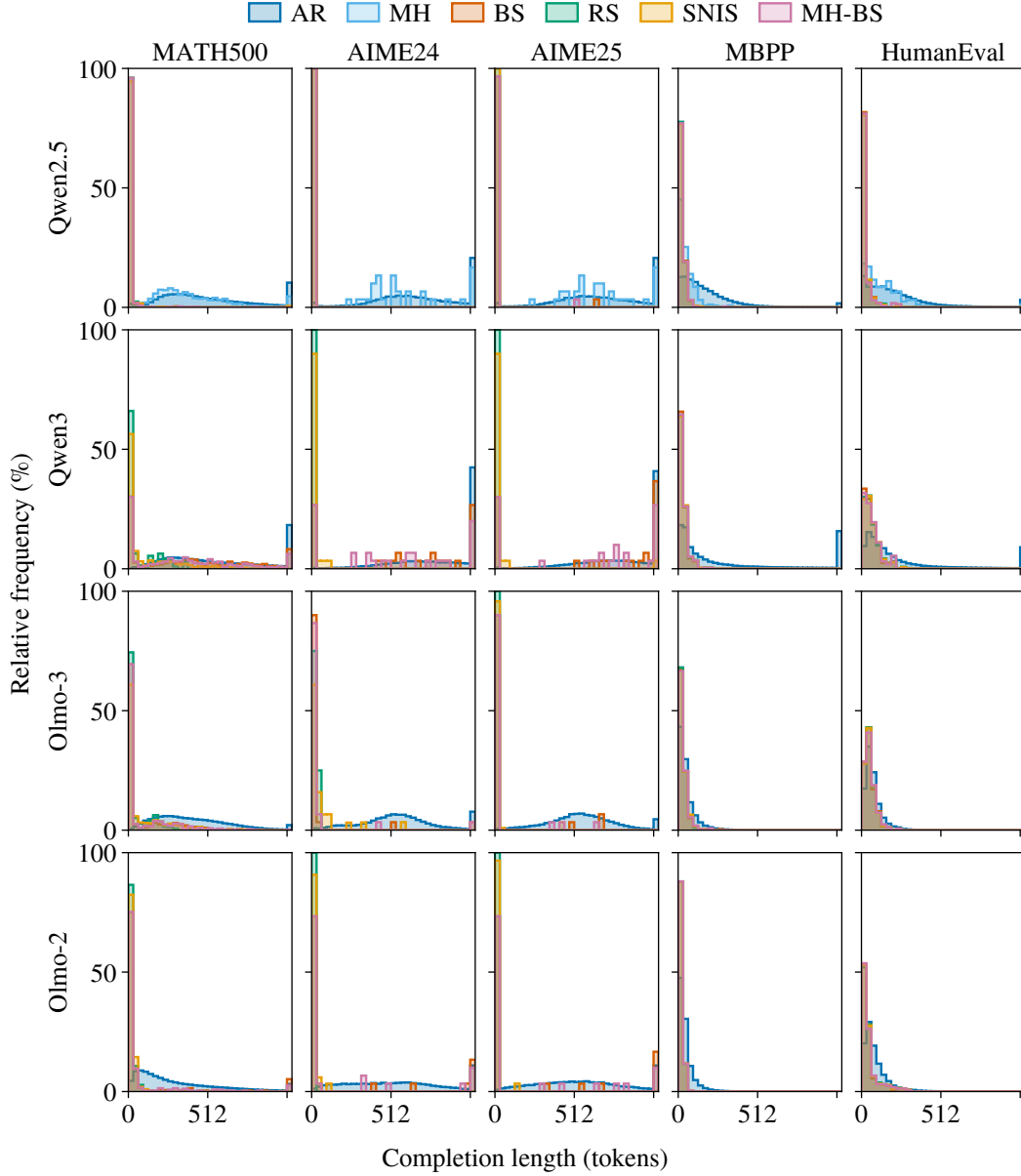


Figure 4: **Completion length under the power distribution.** Completion-length distributions at $\alpha = 4$; the rightmost bin collects completions of at least 1,024 tokens. Rows correspond to models (Qwen2.5-7B, Qwen3-8B-Base, Olmo-3-7B-SFT, Olmo-2-7B-SFT) and columns to benchmarks; method abbreviations as in Table 1. MH (empty initialization) is shown for Qwen2.5-7B only. RS includes only problems with a certified argmax and at least one accepted proposal.

C PROBABILITY MASS OF CORRECT AND DEGENERATE COMPLETIONS

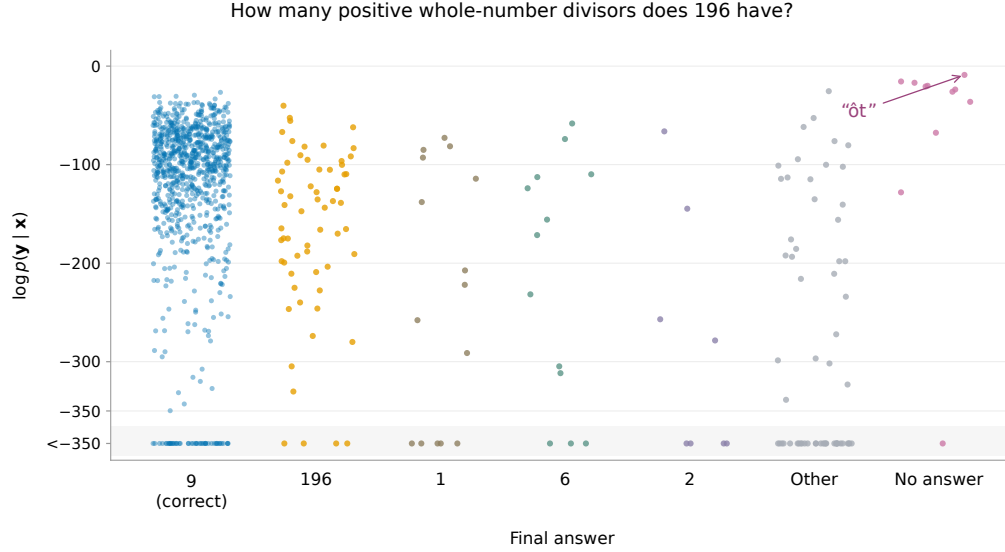


Figure 5: **Individual sequence probabilities for 1,024 temperature-1 Qwen2.5-7B completions.** Each point is a completion, positioned by its final answer and full-sequence log-likelihood. Many completions answer 9 correctly, while rare degenerate outputs have higher individual probability; the annotation identifies the most probable observed completion, “ô”. Horizontal jitter separates points. “Other” contains less frequent answers, and “No answer” denotes outputs without an extractable answer. The shaded bottom strip retains individual points with $\log p(\mathbf{y} | \mathbf{x}) < -350$.

Algorithm 2: Metropolis–Hastings Sampling with Degenerate Initialization**Input** : prompt $\mathbf{x}$; base p ; proposal p_{prop} ; power α ; maximum length T **Hyperparams:** block size S ; MCMC steps N_{MCMC} **Output** : sequence $\mathbf{y}$ approximating a draw from p_α **1 Notation:** With conditioning on $\mathbf{x}$ suppressed, the intermediate target is

$$\pi_L(\mathbf{y}) \propto p(\mathbf{y})^\alpha, \quad \mathbf{y} \in \mathcal{Y}_L.$$

Outputs end at EOS or cap L ; $|\mathbf{y}|$ is the number of tokens before EOS. A chain with $|\mathbf{y}| = 0$ does not move, and a proposal whose new suffix has no token before EOS is rejected.

```

2  $(\hat{\mathbf{y}}, c_0) \leftarrow \text{BestFirst}(p, \mathbf{x}, T)$  // Stage 1 of Algorithm 1
3  $\mathbf{y} \leftarrow \hat{\mathbf{y}}; j_0 \leftarrow \max\{0, \lceil |\mathbf{y}|/S \rceil - 1\}$ 
4 for  $j \leftarrow j_0$  to  $\lceil \frac{T}{S} \rceil - 1$  do
5    $L_j \leftarrow \min\{(j+1)S, T\}$ 
6   if  $\mathbf{y}$  has not ended at EOS then extend  $\mathbf{y}$  with  $p_{\text{prop}}$  until EOS or length  $L_j$ 
7   Use the current state  $\mathbf{y}$  as the initialization for  $\pi_{L_j}$ 
8   for  $n \leftarrow 1$  to  $N_{\text{MCMC}}$  do
9     Sample an index  $m \in \{1, \dots, |\mathbf{y}|\}$  uniformly
10    Construct  $\mathbf{y}'$  by keeping  $\mathbf{y}_{<m}$  and resampling the suffix:
        
$$\mathbf{y}'_t \sim p_{\text{prop}}(\cdot \mid \mathbf{y}'_{<t}), \quad t = m, m+1, \dots$$

        Stop the proposal at EOS or length  $L_j$ 
11    Compute acceptance probability
        
$$A(\mathbf{y}', \mathbf{y}) \leftarrow \min \left\{ 1, \frac{\pi_{L_j}(\mathbf{y}')}{\pi_{L_j}(\mathbf{y})} \cdot \frac{|\mathbf{y}|}{|\mathbf{y}'|} \cdot \frac{p_{\text{prop}}(\mathbf{y}_{m:} \mid \mathbf{y}_{<m})}{p_{\text{prop}}(\mathbf{y}'_{m:} \mid \mathbf{y}_{<m})} \right\}.$$

        Draw  $u \sim \text{Uniform}(0, 1)$ ;
12    if  $u \leq A(\mathbf{y}', \mathbf{y})$  then accept and set  $\mathbf{y} \leftarrow \mathbf{y}'$ 
13  end
14 end
15 return  $\mathbf{y}$ 

```

D METROPOLIS–HASTINGS IMPLEMENTATION

Our Metropolis–Hastings implementation is based on Zuo (2025), an accelerated implementation of the inference algorithm from Karan & Du (2026). We make minor bug fixes to the implementation. The exact procedure is described in Algorithm 15.

E ACCURACY ON PROBLEMS WITH EXACT SAMPLES

Figure 6 restricts the comparison to the problems on which rejection sampling accepted at least one proposal for Qwen2.5-7B, so that every method is evaluated against exact samples on the same problems.

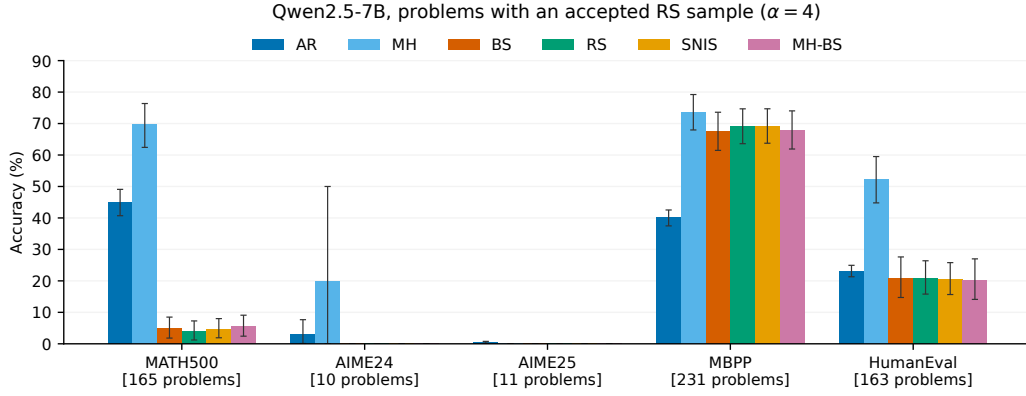


Figure 6: Accuracy (%) of Qwen2.5-7B at $\alpha = 4$ on the problems where rejection sampling accepted at least one proposal; brackets give the number of such problems. Method abbreviations are given in Table 1. Error bars are 95% bootstrap intervals over problems (10,000 resamples, with the same resampled problems for every method).

F IMPORTANCE SAMPLING DIAGNOSTICS

This appendix examines the self-normalized importance-sampling estimates of Section 3.2: how they change with the number of particles, and how many particles effectively contribute to them.

F.1 ACCURACY VERSUS NUMBER OF PARTICLES

Figure 7 shows the estimate of accuracy at $\alpha = 4$ as more of the saved temperature-1 completions are used as particles, adding them in the same fixed order for every particle count. On the code benchmarks, the estimates settle within the first few hundred particles. On MATH500, they still decrease slowly at 1,024 particles for several models. On AIME, each benchmark has only 30 problems, so the estimates change in steps whenever a newly added particle takes over the weight of a problem.

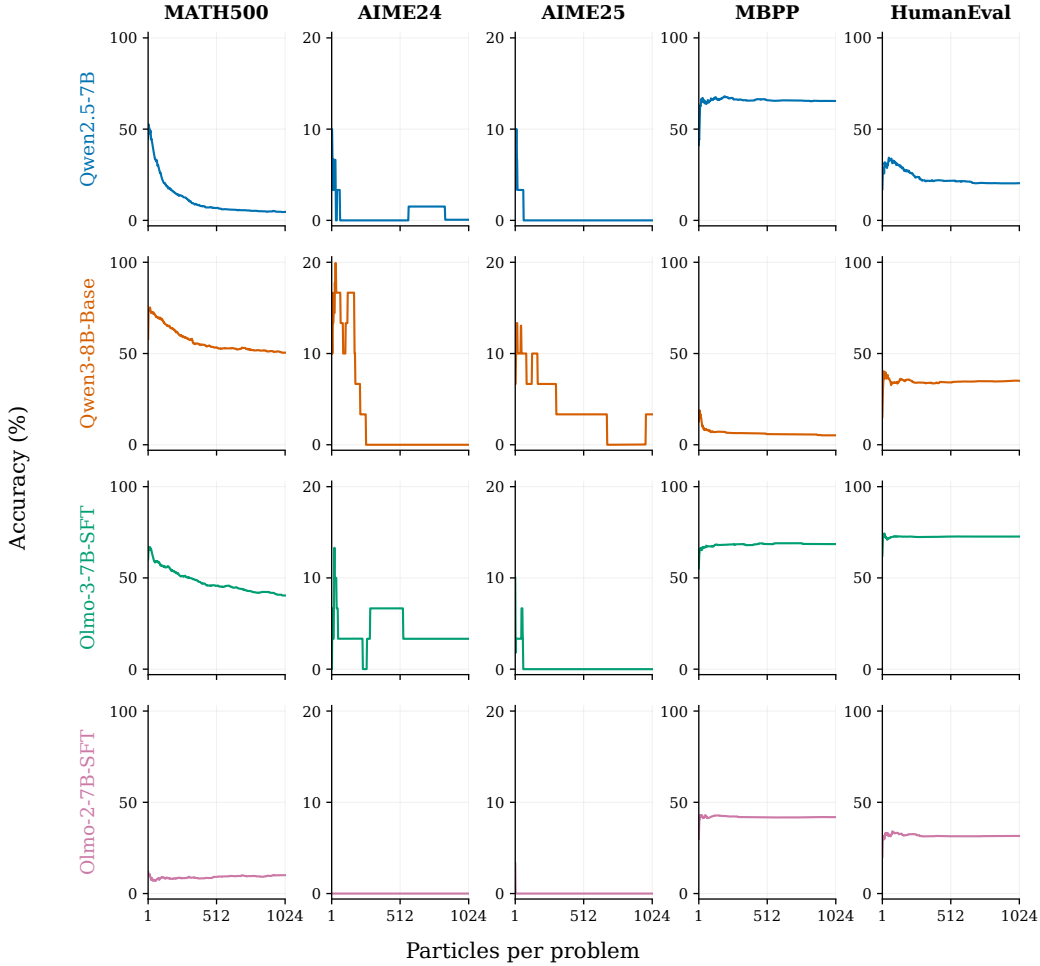


Figure 7: Importance-sampling estimates of accuracy at $\alpha = 4$ as the number of particles increases, for four models (rows) and five benchmarks (columns). For each problem, point N uses saved temperature-1 completions with indices $0, \dots, N - 1$, normalizing weights within the problem before averaging accuracy across the benchmark. Every N from 1 to 1,024 is plotted. AIME panels use a 0–20% vertical scale; other panels use 0–100%. Grading follows Table 1.

F.2 EFFECTIVE SAMPLE SIZE

For each problem, the effective sample size of the self-normalized weights in Equation 4 is

$$\text{ESS}_\alpha = \left(\sum_{i=1}^N \bar{w}_i^2 \right)^{-1}, \quad \bar{w}_i \propto p(\mathbf{y}_i | \mathbf{x})^{\alpha-1}, \quad (8)$$

which ranges from 1, when a single completion carries all the weight, to $N = 1,024$ at $\alpha = 1$. Figure 8 shows its median across problems. The effective sample size falls quickly as α grows. At $\alpha = 4$, its median is between 1.00 and 1.24 on every mathematics benchmark. On the code benchmarks, the median at $\alpha = 4$ is between 8.8 and 167.

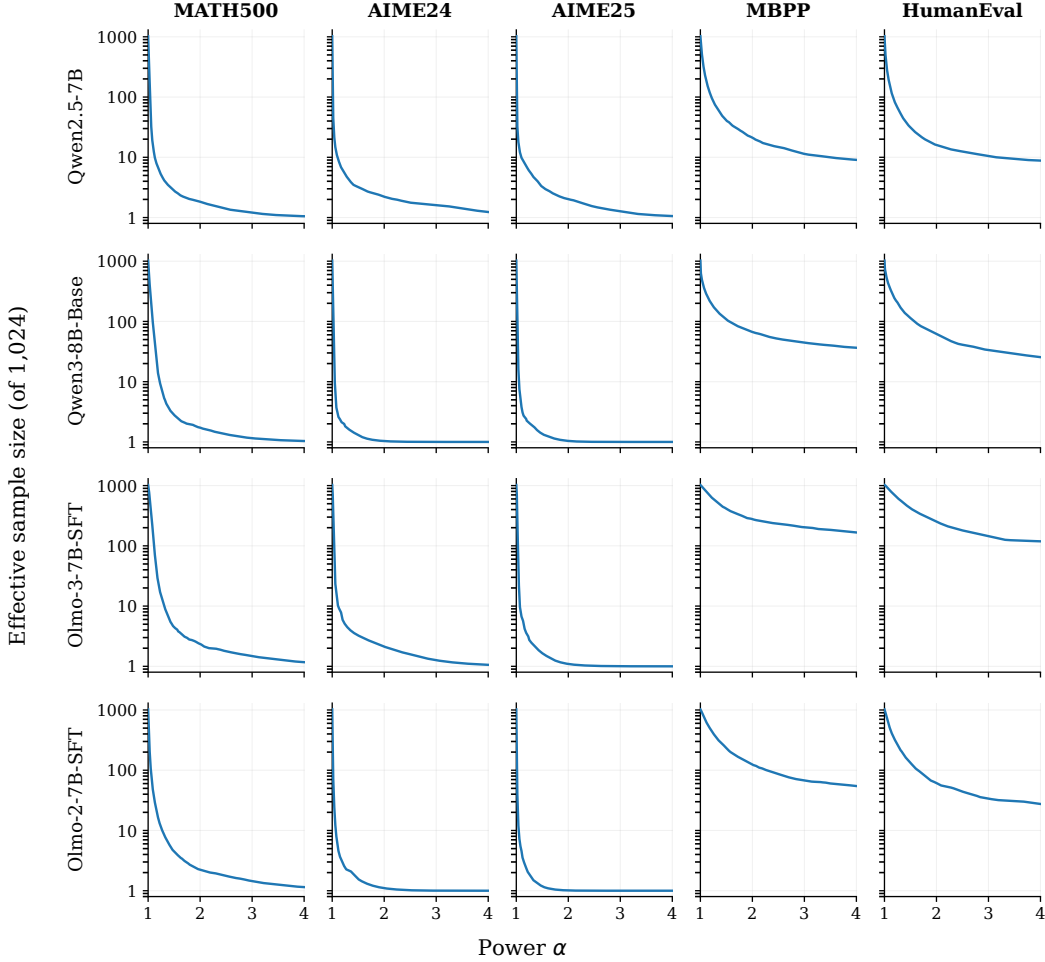


Figure 8: Effective sample size of the self-normalized importance weights as a function of α , for four models (rows) and five benchmarks (columns), using 1,024 temperature-1 completions per problem. Lines show the median across problems. The vertical axis is logarithmic.